

Deepfake Detection Framework: Implementation Roadmap (2025)

Overview and Objectives

This roadmap outlines a phased plan to develop **Reel or Real?**, a web-based deepfake detection and verification framework in Python. The system will provide a **public-facing detection service** with a modern web UI (deployable on Vercel or Heroku) and a scalable backend API (on Google Cloud or AWS). Key objectives include:

- **Multimodal Deepfake Detection:** Use **audio-video analysis** (e.g. Xception CNN, Vision Transformer (ViT), and MFCLIP models) to detect manipulated content. Pre-trained weights will be fine-tuned on a custom dataset (~369 real user videos/images) for real-world generalization.
- **Context-Awareness & Explainability:** Incorporate **contextual cues** and forensic analysis (e.g. irregular eye blinks, lip-sync/emotion mismatches) alongside **explainable AI** outputs like Grad-CAM heatmaps and confidence scores. Metadata provenance checks (e.g. C2PA content credentials) will verify source integrity.
- **Educational Sandbox:** Provide a **private sandbox environment** for users to safely create and experiment with deepfakes. This will help educate users on how deepfakes are made and how the detection works, without enabling public spread of malicious fakes.
- **Modular & Scalable Design:** Architect the system in modular components (data processing, models, explainability, UI) to allow future upgrades – for example, adding physiological signal detectors (heart-rate from video) in later versions.
- **Commercial Viability:** Implement a **3-tier subscription model** (Free, Pro, Business) with feature gating. This ensures a sustainable SaaS offering: a free tier for basic use, and paid tiers with enhanced features (detailed reports, API access, higher usage limits, etc.) for pro users and organizations.

The following phases break down development tasks, milestones, and best-practice guidance, balancing academic depth and real-world product readiness.

Phase 1: Research, Planning & Design (Weeks 1–5)

Key Activities: Conduct foundational research and system design before coding begins. Major tasks in this phase include:

- **Literature Review & Requirements:** Review recent deepfake detection research to identify state-of-the-art methods and gaps. Note that traditional CNN detectors like Xception achieve high accuracy on benchmark datasets but struggle with cross-dataset generalization. Transformer-based models improve temporal modeling but can be opaque. This reinforces the need for a hybrid

approach (CNN + Transformer) with explainability. Compile an annotated bibliography of key papers and document system requirements (functional and ethical).

- **Dataset Survey & Ethics:** Analyze available datasets (FaceForensics++, DFDC, Celeb-DF, KoDF, FakeAVCeleb, etc.) for coverage of different deepfake techniques and modalities. Confirm the core three datasets for training: **DFDC** (for large-scale diverse videos), **FakeAVCeleb** (for audio-visual deepfakes), and **KoDF** (Korean Deepfake Dataset for demographic diversity). Begin paperwork for using the custom user-contributed dataset (~369 real videos/images) – obtain ethics approval and consents, ensuring GDPR compliance for any personal data.
- **Architecture Design:** Draft the system architecture with UML diagrams and flowcharts. Design a **modular pipeline** where components can be developed and tested independently. Key modules identified should include: video/frame processing, audio processing, CNN feature extractor, temporal sequence model (ViT or LSTM/transformer), fusion logic for audio-visual features, explainability module, metadata analyzer, and the web interface. Ensure the design allows adding new detectors (e.g. a pulse detector) without major refactoring.
- **Tech Stack Decisions:** Finalize technology choices for front and back end. On the **backend**, choose Python frameworks like **FastAPI** or Flask for the inference API. FastAPI is preferred for its async support and performance, which will help handle concurrent requests in production. Ensure compatibility with ML libraries (PyTorch for model implementation, plus OpenCV for video processing, etc.). For the **frontend**, a modern JavaScript framework such as **Next.js (React)** is ideal, especially since Vercel is mentioned (Next.js integrates seamlessly with Vercel deployments). This allows building a responsive UI for users to upload videos and view results. Plan for a PostgreSQL or Firebase database to store user accounts, subscriptions, and logs.
- **Cloud & DevOps Planning:** Determine deployment environment – e.g. using **Google Cloud Run or AWS EC2** for hosting the FastAPI inference service with GPU support (NVIDIA T4 or A10G instances for efficient model inference). Outline CI/CD strategy (GitHub Actions or GitLab CI) to automate testing and deployments. Consider containerizing the application with Docker from the start, to ease cloud deployment and scaling later.
- **Risk Assessment:** Identify risks and mitigation strategies. Examples: **computational limits** (mitigate via cloud training or model optimization), **overfitting** (mitigate via data augmentation and cross-validation), and **ethical misuse** (mitigate by not providing public deepfake generation except in a

controlled sandbox with watermarked outputs). Document these along with contingency plans.

Milestones (Phase 1): Completed system design diagrams, finalized dataset and model choices, project proposal document (if academic), and ethical approval submission for user data. The outcome is a clear blueprint and work breakdown for implementation.

Phase 2: Data Collection & Preparation (Weeks 6–9)

Key Activities: Set up the data pipeline – gathering, cleaning, and preparing all datasets for model training.

- **Acquire & Organize Datasets:** Download or collect the chosen public datasets (**DFDC, FakeAVCeleb, KoDF**). Ensure a consistent directory structure (e.g. videos split into train/val/test sets). Where needed, convert videos to a standard format (e.g. MP4, consistent resolution). If the custom user dataset is approved, ingest those ~369 real videos/images, tagging them appropriately (they may serve as additional “real” samples to improve authenticity checks). Balance data from multiple sources to avoid bias – e.g. ensure deepfakes and real samples are roughly equal per batch.
- **Preprocessing Scripts:** Implement scripts to **extract video frames and audio** from each video. For example, sample a certain number of frames per video (perhaps 1–5 frames per second of video) to reduce redundancy while preserving key content. Store frames as JPEGs and audio as WAV files or mel-spectrogram images for input to models. Compute **Mel-Frequency Cepstral Coefficients (MFCCs)** from audio using librosa or a similar library. MFCCs will serve as features for audio-based deepfake detection, as they capture vocal tract characteristics that can reveal voice cloning.
- **Data Augmentation:** To improve generalization, apply augmentations to training data. For images/frames: random flips, rotations, color jitter, gaussian noise, or JPEG compression to simulate various video qualities. For audio: slight time stretching or adding background noise can help the model not overfit to a specific audio quality. Augmentations address dataset bias (e.g. DFDC deepfakes may have certain artifacts that augmentation can smooth out).
- **Annotation & Metadata:** If needed, create labels/annotations. Deepfake datasets usually come with labels (real or fake). Double-check any edge cases (e.g. remove or label any partially manipulated or uncertain samples). Also, **extract metadata** from videos/images at this stage – e.g. using ExifTool or ffmpeg to parse metadata fields (camera model, encoding software, timestamps). This metadata will later feed into provenance analysis. Note if any videos contain **C2PA content credentials** or watermarks in metadata. (While

rare in current datasets, the system should be designed to detect a C2PA manifest indicating an authentic asset.)

- **Dataset Splits & Validation:** Split data into training, validation, and test sets. Ensure that deepfake footage of a particular subject appears only in one set to test generalization. Use stratified splitting by source if possible (e.g. ensure each dataset's content is distributed so the model doesn't just memorize one dataset's specific artifacts). For the user-contributed real videos, keep them mostly in training for model fine-tuning, but reserve some for a **“real-world” test set** to evaluate performance on genuine, previously unseen content.

Milestones (Phase 2): All datasets are prepared and loaded into storage accessible by the training pipeline. We have a **unified data loader** that can feed video frames and audio features into models. The data is verified (no corrupted files, labels correct) and initial data analysis is done (e.g. verify that deepfake videos exhibit known artifacts like visual blurs or audio inconsistencies). The project can now progress to model development with confidence in the data foundation.

Phase 3: Model Development – Baseline Models (Weeks 10–14)

Key Activities: Implement baseline deepfake detection models for visual-only and audio-only modalities. This provides a starting point before full multimodal fusion.

- **Visual Deepfake Model (CNN):** Start with a proven CNN architecture such as **Xception** (which was top-performing in early deepfake detection benchmarks). Implement the Xception network (e.g. using PyTorch and loading ImageNet pre-trained weights for transfer learning). Replace the final layer to output a binary classification (real vs fake). Train this model on **video frame data** (treating each frame as an independent image sample initially). Use the training split of frames from deepfake videos vs real videos, and optimize binary cross-entropy loss. Monitor validation accuracy and AUC on a hold-out set (e.g. from DFDC). This CNN will capture **spatial artifacts** in single frames (like warped facial textures, missing reflections, etc.) and serve as a baseline. Expect that it performs well on the data it's trained on, but we'll later address temporal cues and generalization.
- **Temporal Sequence Model (ViT or Conv-LSTM):** To capture **temporal inconsistencies**, develop a model that processes sequences of frames. One approach is using a **Vision Transformer (ViT)** adapted for video – e.g. a TimeSformer or ViViT model that can take a series of frames and use attention to model changes over time. Transformers can learn temporal attention patterns (e.g. sudden unnatural changes in facial features across frames). If implementing a full transformer for video is complex, a simpler approach is a **ConvLSTM or 3D CNN** (like a 3D Xception) that extends CNN kernels across time. For a first baseline, one could feed sequences (say 8–16 frames) into an

LSTM that takes CNN feature vectors as input. **Training:** possibly first train the CNN to convergence, then freeze it and train an LSTM on top of its features to predict real/fake for the sequence. This two-stage approach (CNN then LSTM) is a form of hybrid model that has shown improved detection of manipulations in videos.

- **Audio Deepfake Model:** In parallel, develop an audio-based detector. A simple baseline is to use the extracted **MFCC features** and train a classifier (e.g. a small CNN that treats MFCC spectrogram as an image, or an LSTM over the MFCC time series). Alternatively, use a pre-trained audio deepfake detection model if available (for instance, models from the ASVspoof challenge or a pretrained CNN on spectrograms that can detect synthesized voices). The audio model aims to catch signs of voice tampering: artifacts from text-to-speech, robotic intonations, or mismatched background noise. Train it on the audio tracks of the same dataset videos (FakeAVCeleb is especially useful here as it has both genuine and fake audio). Ensure the audio model outputs a probability of fake vs real for a given clip's sound.
- **Preliminary Evaluation:** After training these baseline models, evaluate them on a **validation set and a small test set**. For example, measure the CNN's accuracy on KoDF or DFDC test videos (frame-level metrics and aggregate video-level metrics). If the CNN overfits to known deepfakes, consider techniques like **regularization and mixup** to improve robustness. Evaluate the audio model on a subset of FakeAVCeleb audio: it might identify obvious fake voices but could confuse background music or poor recording quality as fake (false positives). Note down these observations for improvement in the fusion phase.
- **Iterate Model Tuning:** Use the validation results to iterate. For instance, if the CNN's performance plateaus, consider fine-tuning more layers or using a slightly different architecture (e.g. EfficientNet or a ResNet50) to compare. If the ViT-based approach is too slow or not outperforming, a potential adjustment is to use **frame-level CNN + temporal averaging** (i.e. average the CNN predictions over frames of a video as a quick video-level prediction baseline). These experiments set a performance baseline to beat with the integrated approach.

Milestones (Phase 3): Achieve functioning **baseline detectors**: one that analyzes images/frames (spatial model) and one for audio. We should have baseline accuracy (e.g. the Xception might achieve decent accuracy on DFDC but could be lower on KoDF if it's out-of-distribution – documenting this is important). This phase produces saved model weights for use in the next phase and establishes a reference point (e.g. "CNN-only AUC = X, audio-only AUC = Y"). The system also now has the basic inference code to input a video frame or audio and get a prediction. These will be integrated next.

Phase 4: Multimodal Fusion & Advanced Modeling (Weeks 15–20)

Key Activities: Develop the integrated audio-visual deepfake detection model and improve context awareness with advanced techniques. This phase aims to boost detection robustness by combining modalities and leveraging cutting-edge models like MFCLIP.

- **Audio-Visual Fusion Model:** Design a model that **fuses features from both video and audio** to produce a joint prediction. A straightforward approach is **late fusion**: use the trained visual model and audio model from Phase 3 to output intermediate features or probabilities, then train a shallow neural network to combine them. For example, take the penultimate layer embedding of the CNN+Transformer (video) and the embedding from the audio model, concatenate them, and train a small feed-forward layer (or even use a weighted average of probabilities) to predict real/fake. Train this fusion on a subset of videos that have both modalities. Ensure that during training, the network sees matched pairs (video + audio from the same clip). This will teach the fusion model to detect inconsistencies between modalities (e.g. the video says one thing while audio tone/emotion says another). **Alternative:** Explore an **early-fusion transformer**: feed both visual frame sequence and corresponding audio features into a multimodal transformer where cross-attention can learn correlations (this is more complex and computationally heavy, but some research suggests cross-modal attention can catch subtle mismatches).
- **Incorporating MFCLIP:** Research the recently proposed **MFCLIP (Multi-modal Fine-grained CLIP)** approach, which uses a CLIP-based architecture to find forgery traces across image and noise modalities. While implementing MFCLIP from scratch might be beyond the scope of a single project, we can borrow ideas:
 - Use a **pre-trained CLIP model** to encode video frames (image encoder) and perhaps audio spectrograms or even frame-level high-frequency noise patterns (text encoder with prompts as done in MFCLIP). The idea is to leverage a foundation model that has seen huge data, for better generalization to new deepfakes.
 - If time permits, fine-tune this CLIP-based setup on a subset of our data to see if it catches artifacts the CNN misses. MFCLIP specifically targets diffusion-based fakes and noise patterns; integrating a simplified version (like adding a frequency-domain analysis branch) could future-proof the system against emerging deepfake generation techniques.
- **Training & Fine-Tuning:** Fine-tune the multimodal model using a curriculum:

1. Start by training on **synchronized fake videos** (e.g. from FakeAVCeleb or DFDC, where both audio and video are fake in tandem).
 2. Then challenge the model with **mixed fakes** – for example, pair real video with fake audio and vice versa. This will teach the model to recognize incoherence (e.g. mouth movements not matching speech). Create some mixed pairs manually if needed: replace audio of a real video with audio from a fake or a different person to simulate an audio-deepfake scenario.
 3. Finally, evaluate on truly difficult cases (like real videos dubbed in another language, which are technically not deepfakes but have mismatched audio-visual content) to ensure the system doesn't over-flag those. Use these to adjust the fusion thresholds or logic (perhaps require both modalities to be convincing fakes to label as fake, or output a warning for partial mismatches).
- **Performance Optimizations:** As models are now heavier (especially if using a transformer or ensemble of CNNs), profile the inference speed. Apply techniques like model quantization or knowledge distillation to keep the model lightweight for real-time use. For example, distill the knowledge of the ensemble (Xception + ViT + audio net) into a single smaller model if possible. Alternatively, plan to deploy with a GPU on the server to handle the load. **Batching:** ensure that if the front-end sends multiple videos, the backend can process frames in batch for efficiency.
 - **Validation:** Evaluate the fused model on all test sets: it should outperform the single-modality baselines (e.g. higher AUC and F1 on videos where either audio or video alone might have been spoofed). Pay special attention to **generalization:** test on a dataset not seen in training (like if not trained on KoDF, test on KoDF) to verify the context-aware approach isn't overfitting to one dataset's artifacts. Research indicates multimodal integration and transformers can improve generalization across datasets. Document results and examples of successes (e.g. the model caught a deepfake that only changed audio because it noticed emotional mismatch) as well as failures (e.g. maybe it flags a foreign language video as fake due to unexpected audio – this could guide adding a language-aware component or threshold adjustments).

Milestones (Phase 4): A **unified multimodal deepfake detector** is built and tuned. We have model weights and code that, given a video file, returns a prediction of real/fake that considers both the visual and audio evidence. Internally, the approach now integrates both CNN-based spatial analysis and transformer-based temporal analysis, addressing known gaps (poor cross-dataset generalization and limited multimodal fusion). We expect a notable improvement in tricky cases – for instance, detecting

deepfakes that only an audio or visual model alone might miss. This sets the stage to add explainability and deploy the model in an interactive service.

Phase 5: Explainability & Forensic Analysis Integration (Weeks 21–24)

Key Activities: Enhance the system’s transparency and trustworthiness by integrating explainable AI features and additional forensic checks (contextual indicators of fakeness). This phase ensures the model’s decisions can be understood by users and that the system provides richer analysis beyond just “real” or “fake.”

- **Grad-CAM Visual Explanations:** Implement **Grad-CAM** for the CNN component of the model to highlight *where* the model sees potential manipulation. For each input video frame analyzed, compute Grad-CAM heatmaps on the final convolutional layers of Xception (or the CNN backbone of the ViT, if applicable). This will produce an overlay showing, for example, if the model focuses on the eyes, mouth, or edges of the face when predicting “fake”. Integrate this into the inference pipeline so that along with a prediction, the system outputs a set of heatmap images. These can be averaged or sampled across a video to show consistent artifacts (e.g. “the model found the mouth region suspicious in many frames”). In the UI, these heatmaps will be displayed to the user to explain the result (e.g. highlight a strange cheek distortion or unnatural eye area that led to a fake classification).
- **Transformer Attention Maps:** For the temporal model (if using a Transformer or even the audio-visual cross-attention model), extract **attention weights** or similar indicators to explain sequence-level reasoning. One approach is to visualize which frames the transformer deemed most important for its decision. Alternatively, if using an LSTM, show which time segments had high model error. This can translate to highlighting moments in the video where anomalies occur (e.g. a frame where the face blending glitches, or a moment the voice suddenly has a different tone). While attention maps are more complex to present, we can simplify by perhaps picking the top-attended frames and marking them as “key questionable frames” to review.
- **Confidence and Uncertainty:** Alongside the binary verdict, compute a **confidence score** (e.g. the model’s output probability for “fake”). Display this as a percentage or a gauge, so users understand the certainty. Additionally, implement basic uncertainty estimation – for example, run the model with slight input perturbations or use dropout at inference (Monte Carlo dropout) to see if predictions vary. If uncertainty is high (confidence oscillates), the system can flag the result as less reliable. Communicate this on the front-end (e.g. “The result is 60% confident; consider additional verification”). This encourages responsible use by not over-claiming accuracy.

- **Facial Forensic Checks:** Integrate **specialized detectors for known deepfake artifacts**:
 - **Eye Blink Analysis:** Deepfake videos have historically shown irregular or absent blinking. Implement an eye-blink detector: use OpenCV or a lightweight model to track the subject's eye state across frames, computing the blink frequency and duration. Compare this to typical human blinking patterns. If the subject hardly blinks or blinks in an unnatural rhythm, flag it. This can be quantified (e.g. "blinks per minute = X, which is abnormally low"). This measure will be included in the report or even as a feature input to the model in future iterations.
 - **Lip-Sync and Emotion Consistency:** Compute a simple **lip synchronization score** by comparing mouth movement (e.g. via lip landmarks or an existing lip-sync error model like *SyncNet*) to the audio track. Large discrepancies (speaking without corresponding mouth movement or vice versa) indicate tampering. Similarly, perform **emotion analysis** on both modalities: use an off-the-shelf facial emotion recognition (FER) model on video frames to get the expressed emotion (happy, angry, neutral, etc.), and use either an audio emotion classifier or even text sentiment (if speech can be transcribed) for the spoken content. If the dominant emotions don't align (e.g. video face looks calm but the voice tone is angry), that's an **emotion mismatch** suggesting the audio may not belong to the video. Research supports that such multimodal sentiment inconsistency is a strong deepfake signal. Implement this check and include an "emotion coherence" indicator in the output.
 - **Other Forensics:** If time permits, add checks for **head pose** (weird head movements or face angle inconsistencies) and **facial distortion** (using facial landmarks to see if geometry stays consistent). These can be simple heuristic checks reported to the user (e.g. "Warning: The face geometry is inconsistent across frames").
- **Metadata Provenance Analysis:** Integrate a module to analyze the **metadata and provenance** of the submitted content. Specifically, implement reading of any **C2PA manifests** in the file (using a library or the standard, if available). If a valid content credential is present, the system can inform the user of the asset's origin (e.g. "This video is certified by X camera or X publisher as authentic, with no edits"). If the deepfake model flags something as fake but the provenance data says it's original, it warrants a closer manual review – include guidance in the interface for these scenarios. Also, check for simpler metadata clues: for instance, if the video's metadata shows software like *Adobe Premiere* or *DeepFaceLab* in the encoder fields, that could indicate tampering. List any

detected editing software, and highlight if metadata was **stripped** entirely (since deepfakes often have minimal metadata). Additionally, consider **digital watermark detection**: some generators or stock footage may include invisible watermarks (though a custom solution for this might be complex, so focus on standard metadata and C2PA).

- **User Interface & Visualization:** Modify the front-end to present these new explainability features. For example, when a user gets a result, they should see:
 - A label Real or Fake with confidence %.
 - An expandable **“Evidence” section** showing a representative frame with a Grad-CAM heatmap overlay, and perhaps a short looping clip of the moment with irregularities (if lip-sync or blinking issues were found, highlight that segment).
 - A summary of forensic checks: e.g. “Blink rate: very low (possible deepfake); Audio-visual emotion mismatch detected; No camera metadata found (content may have been edited)”.
 - A disclaimer about the system not being 100% accurate and encouraging user judgment (responsible AI practice). If any C2PA verification passed or failed, display that (e.g. green check if content is verifiably authentic, or red X if an expected signature is missing/corrupted).

Milestones (Phase 5): The deepfake detection service becomes **explainable and context-rich**. By the end of this phase, the backend can output not just a classification, but a detailed JSON (for example) containing the result, confidence, and various analyses (heatmaps saved as image paths, metrics like blink count, any metadata flags). The frontend is updated to beautifully visualize this information, turning the tool into an educational experience for users. We will have effectively addressed the interpretability and user-trust gap that many AI detectors suffer. A test with a few sample videos can demonstrate how the system flags fakes with explanations (e.g. a deepfake video of a public figure where the system highlights odd eye rendering and missing metadata, explaining its reasoning).

Phase 6: Frontend Development & Sandbox Environment (Weeks 25–28)

Key Activities: Develop the user-facing components – the web application interface and the deepfake generation sandbox – and integrate them with the backend API. Although some frontend work started earlier (for visualization), this phase focuses on delivering a polished, deployable web service.

- **Frontend Web App (UI/UX):** Using **Next.js/React**, implement a clean interface with intuitive workflows. Key pages/components:

- **Home/Detection Page:** Where users upload a video or image for analysis. Include drag-and-drop or file selector for input. Show processing status (maybe a spinner or progress bar while the backend analyzes, since this could take ~several seconds for a video). Once results are ready, display the detection results and evidence (as structured in Phase 5). Emphasize clarity: use charts or icons for confidence (e.g. a gauge or percentage). The heatmap can be shown by overlaying on the video frame canvas in the browser; use a library like D3 or just canvas API to blend the heatmap. Allow toggling evidence layers (some users may just want the verdict, others may want all details).
- **Sandbox Page (Deepfake Creator):** This section (likely gated for logged-in users, see subscription tiers) allows a user to create a deepfake **privately**. Provide options depending on available generation capabilities: e.g. *Face swap* between two images or a video and an image, *Voice cloning* by inputting some audio, etc. For simplicity, perhaps implement one use-case: **Face swap video** – user uploads a source video of themselves and a single image of a target face (or pick from a set of example faces). Then on the backend, run a face-swapping model (there are open-source models like SimSwap, FaceSwap, or First Order Motion Model for reenactment). Because training a new GAN is infeasible, use a pre-trained generator if available (maybe integrate with a tool like *Avatarify* or *Wav2Lip* for lip syncing). The backend will generate the fake and return it. In the UI, show a video player of the output. Clearly watermark the output (e.g. a semi-transparent “DEEPFAKE (sandbox)” text on the video) to prevent misuse. Also, do not allow sharing directly from the app – the content stays in the user’s session unless they explicitly download it, and warn them to use responsibly.
- **Login/Account Management:** Implement user authentication. This can use a provider (Auth0, Firebase Auth) or a custom solution with JWTs. At minimum, users should create an account with email to access Pro or Business features. Free tier might allow anonymous or limited use. The account dashboard should show their plan, usage (e.g. how many videos analyzed this month), and perhaps billing info for paid tiers.
- **Admin/Monitoring (internal):** If building a startup, include admin views or at least the backend ability to monitor usage, view logs, and manage content (e.g. if someone uploads illegal content, it should be flagged – though that’s beyond scope, but content policy should be considered in responsible deployment).

- **Integrate Frontend with Backend API:** Develop the REST API endpoints in the FastAPI backend to handle requests:
 - **/analyze (POST):** Accepts an uploaded video/image, plus maybe user auth token. The backend will run the deepfake detection pipeline (Phase 4 & 5 output) and return a JSON response with all results (verdict, confidence, explanations, etc.) and possibly URLs or base64 data for heatmap images. The frontend then processes this JSON and displays the results nicely. Ensure to handle large files (maybe limit upload size and duration on free tier). Use asynchronous processing if needed – e.g. if analysis takes long, the API can immediately respond with a job ID and the frontend polls for result. For simplicity in MVP, a synchronous call with a reasonable timeout (30-60s) can be done, but for production, consider a background task queue (Celery or RQ) to handle heavy jobs and avoid request timeouts.
 - **/generate (POST):** (if sandbox generation) Accepts required inputs (could be multiple files) for the generation task. Since generation may take time (especially if using a deepfake model on CPU), handle it similarly with either a longer wait or a background job. Return the generated media (or a link to download it). The sandbox generation should only be accessible to certain users (check API key or auth scope for Pro/Business tier).
 - **/login, /signup, /subscription:** If not using an external auth, provide endpoints for authentication and subscription management. This could integrate with Stripe or another payment API for upgrading accounts (see Phase 7). However, to keep within scope, basic login and checking user tier in each request (via JWT or session) is sufficient.
- **Testing UI/UX:** Perform user testing on the interface with a few peers or pilot users:
 - Is the upload and result display smooth?
 - Do users understand the explainability visuals and forensic info provided? (e.g. maybe add tooltips explaining terms like “C2PA metadata” or “MFCC audio features” in layman’s terms).
 - Test the sandbox with benign content to ensure it works and the output is clearly marked. Confirm that sandbox output, if put back into the detection tool, is indeed flagged as fake (this is a neat feedback loop to demonstrate the detector’s effectiveness!).

- Responsive design: ensure it works on different devices (perhaps limit to desktop for heavy video features, but at least mobile-friendly layouts for results text).

Milestones (Phase 6): The **full web application** is ready in a staging environment.

Users can interact with the system: upload a video to get a deepfake verification report, and (if authorized) create a deepfake in the sandbox. The integration between frontend and backend is working, with proper error handling (e.g. if a user uploads an unsupported format or a very large file, the UI shows a meaningful message). By now, we should have a compelling demo: a friendly user interface layered over a sophisticated detection engine. This meets the academic goals (demonstrating a working prototype with novel features) and sets up the product foundations (user accounts, web delivery).

Phase 7: Testing, Evaluation & Iteration (Weeks 29–32)

Key Activities: Rigorously test the entire system for accuracy, performance, security, and user satisfaction. Use feedback to refine the models and UX before a public launch.

- **Quantitative Evaluation (Accuracy & Performance):** Using held-out test data and some real-world samples, evaluate the detection performance:
 - Compute metrics like **accuracy, precision, recall, F1-score, AUC** on each test dataset (DFDC, KoDF, FakeAVCeleb). Also test cross-dataset (e.g. train on DFDC/FakeAVCeleb, test on KoDF) to see generalization. Ideally, the multimodal model should show improved generalization (higher recall on KoDF than a CNN-only baseline, for instance).
 - Evaluate robustness: test on videos with compression, noise, or partial deepfakes. For example, take a real video and insert a single fake face frame – does the system catch it or does averaging dilute the detection? Check robustness against common transformations (resize, format change) by passing altered videos to the detector and ensuring results don't fluctuate wildly.
 - **Speed and Scalability:** Measure inference time per video (for various lengths/resolutions) on the chosen hardware. If a single analysis for a 10-second video takes too long (>30s) on a GPU, consider optimizing further or note it as a limitation for the free tier (perhaps long videos reserved for paid tiers or cut into segments). Test concurrent requests to simulate multiple users – does the system slow down or queue properly? Use load testing tools if needed.
- **Qualitative Evaluation (User Study):** Conduct a small user study for feedback on the system's explainability and usefulness. Select ~5-10 participants (could

be classmates or target users like journalists or content creators) and have them try the tool on a few videos. Gather feedback via a survey:

- Do they find the explanations (heatmaps, flags) clear and helpful in understanding why something is fake?
- Does the interface present the information in a non-confusing way?
- How likely would they be to trust the system's result, or use it in real scenarios?
- For the sandbox: did it help them learn about deepfakes? Was it easy to use? (Ensure none abused it – remind it's for learning and outputs are watermarked).

Analyze qualitative feedback and identify common pain points or suggestions. For instance, users might say “I wish the tool gave an overall score instead of just fake/real” or “The term ‘MFCC’ was jargon to me” – these can be addressed by UI tweaks (like adding a summary sentence: “Overall Deepfake Likelihood: High”) or adding a glossary/help section.

- **Security and Ethics Review:** Before deployment, do a check for security issues:
 - Ensure uploaded files are stored securely or deleted after analysis (don't keep user videos longer than necessary, given privacy concerns).
 - Protect the API endpoints (rate limiting to prevent abuse, authentication required for certain actions, sandbox generation perhaps with additional verification to prevent misuse).
 - If possible, run some basic penetration tests or use static analyzers for the code. Also, review compliance with data protection (if any user data is stored, ensure passwords hashed, etc.).
 - Ethical use: confirm the system cannot be easily repurposed to create deepfakes at scale (the sandbox is limited and outputs are clearly marked). Also provide **Terms of Service** and a disclaimer in the UI that misuse of the service (e.g. uploading non-consensual images for deepfakes) is prohibited. This manages legal risks.
- **Iterate Improvements:** Based on evaluation, loop back to earlier phases as needed:
 - If certain deepfakes were missed (false negatives), examine why. Maybe add those cases to training or adjust the decision threshold.
 - If certain real videos were misclassified (false positives), consider if it was due to unusual content (e.g. a real video with a filter might look fake).

Possibly add more context checks or adjust model bias. Utilizing the **bias evaluation** techniques (like those in latest research) might be beyond our scope, but keep an eye on any demographic bias (e.g. does it work equally well on different ethnic groups as per KoDF diversity? If not, more training data from that group or fine-tuning might be needed).

- Refine the UI/UX per user feedback. Maybe simplify certain info or add a feature like allowing users to download a PDF report of the verification (which could be useful for journalism or legal use-cases).
- Verify the subscription gating is working (simulate a free user vs pro user to ensure features toggle correctly).

Milestones (Phase 7): The system is **tested and refined**, ready for real use. We have documented evaluation results to include in the final project report (if academic, this is crucial to demonstrate success metrics). Also, we likely have a batch of example deepfake videos and their analysis results to showcase (perhaps in a portfolio or demo site). The end of this phase should yield confidence that the framework is accurate, user-friendly, and secure enough to deploy widely.

Phase 8: Deployment & Launch Strategy (Weeks 33–36)

Key Activities: Deploy the application on cloud infrastructure, implement the subscription model with pricing, and prepare for a production launch as a SaaS offering.

- **Cloud Deployment (Backend API):** Containerize the backend (if not done) and deploy to a scalable service. For example, build a Docker image for the FastAPI app including the ML models. Use **Google Cloud Run** for deployment – it can run the container with automatic scaling; importantly, enable GPU support if using GPU (GCP has Cloud Run for Anthos or use Google Compute Engine with a custom setup for GPU). Alternatively, use an **AWS EC2 GPU instance** or AWS ECS with a GPU-enabled task. Ensure environment variables or config for connecting to the database (for user data) and any API keys (for payment or analytics). Set up **HTTPS** and a custom domain (e.g. reel-or-real.ai). Also configure logging/monitoring (use CloudWatch or Stackdriver to monitor API latency, errors, etc.).
- **Frontend Hosting:** Deploy the Next.js app on **Vercel** (which is optimized for Next.js). Vercel will handle CDN and scaling for the frontend. Configure it to point API requests to the backend's URL. Alternatively, if using a unified deployment, host the frontend on a static storage or the same domain via a reverse proxy. Ensure that the static assets (JS, CSS) load quickly and use caching. Also, set up a global error handler – e.g. if the backend is down, the frontend should show a user-friendly message rather than just failing.

- **Database & Storage:** Launch a managed **PostgreSQL** instance (on AWS RDS or GCP Cloud SQL) for production, to store user accounts, hashed passwords, subscription status, and usage logs. Also allocate a cloud storage bucket (S3 or GCP Storage) for storing uploaded videos temporarily and any generated outputs or reports. Use signed URLs or secure access so that only authorized users can access their own files. Implement retention policies (maybe auto-delete uploaded content after a certain time to save space and protect privacy).
- **Subscription Model Implementation:** Integrate a payment and subscription management system. A common approach is using **Stripe** for billing. Define the **three tiers**:
 - **Free:** Users can analyze a limited number of videos (e.g. 5 per day) with basic results. Possibly lower file size or length limits, and no access to sandbox. Free tier might also have lower priority in processing (if queueing tasks). This tier is to attract users and offer basic utility at no cost.
 - **Pro:** A paid individual plan (e.g. \$X/month) that includes a higher usage quota (say 50 videos per month or longer videos allowed), **full explanation features** (all the Grad-CAM and forensic details), and **access to the sandbox** for personal experimentation. Pro users might also get faster processing or the ability to upload slightly longer videos.
 - **Business/Enterprise:** A higher-price plan (e.g. \$Y/month or custom pricing) aimed at organizations (newsrooms, social media companies, etc.). This includes **API access** for integration (so a business can send videos programmatically to the API and get results, useful for automating content verification). Also, higher limits (perhaps unlimited or very high number of analyses, priority support, and possibly on-premise deployment options for enterprise deals). Business tier could allow multiple team members under one account, and a dashboard to monitor usage across the team.

Each tier should have clear **feature gating** in the code: e.g., an `is_pro` flag might unlock the sandbox UI and higher API rate limits. Use middleware in the API to check the user's plan before processing requests. For example, a free user's request beyond the daily limit returns a polite error suggesting upgrade. Use Stripe's webhooks to upgrade/downgrade user status in the database when payment info changes. Include a **trial period** perhaps for Pro features to encourage upgrade. This tiered strategy aligns with SaaS best practices (similar to Slack's Free/Pro/Business+ model).
- **Pricing Strategy:** Research market rates for similar services (if any exist – deepfake detection SaaS is emerging). The price must balance accessibility and

covering compute costs. For instance, inferencing on a GPU has a cost, so estimate cost per video (maybe a few cents each for a short video). Ensure the Pro/Business pricing covers heavy usage. Possibly implement **overage or usage-based** pricing for enterprise (e.g. a base subscription plus a fee per 100 videos beyond some limit). Keep the free tier sufficiently useful but limited to funnel serious users into paid plans.

- **Launch and Promotion:** With the product ready, plan the launch. If this is part of a final-year project, the “launch” might just be deploying for examiners/university usage. For a real startup launch, prepare documentation (user guide, API docs for business tier), a marketing website explaining the value (e.g. “Deepfake detection powered by multimodal AI and explainable results”), and perhaps a blog post or demo video. Emphasize the unique features: context-aware detection, educational sandbox, and the transparency of the system (these differentiate it from black-box detectors). Possibly engage with the Responsible AI community or content authenticity initiatives (like C2PA’s community) to build credibility.

Milestones (Phase 8): The application is **live in production** under the 3-tier model. We have real users signing up, or at least a staging environment that mimics production settings. All components (frontend, backend, database, payment) are working together. We have monitoring in place to catch any errors. The final deliverable of this phase might include a presentation or pitch, treating the project as not just an academic exercise but a viable startup product. We also have a plan for scaling: e.g. if user growth increases, we can add more backend instances or upgrade the database.

(At this point, the core development is complete. The remaining phase is about long-term maintenance and future improvements.)

Phase 9: Maintenance and Future Enhancements (Beyond initial launch)

Though outside the immediate scope of the project timeline, it’s important to consider future steps to **sustain and enhance** the framework in line with best practices and evolving threats:

- **Monitoring and Updates:** Continuously monitor system performance in the real world. Set up alerts for model drift – e.g. if a sudden spike in false positives/negatives is observed (perhaps due to a new type of deepfake emerging). Plan to periodically retrain or fine-tune the models as new data becomes available. For example, integrate new deepfake examples generated by advanced GANs or diffusion models that come out in 2026, and update the model to detect those. Maintain a feedback loop: if users report a video that fooled the detector, use that as a new training example (with proper consent).

- **Physiological Signal Detection:** Integrate **pulse detection (rPPG)** to catch deepfakes via subtle heartbeat signals in videos. Research like Intel's FakeCatcher has shown that real videos have detectable blood flow changes in faces, which deepfakes usually lack. A future module could analyze color variations in skin (especially around cheeks) to estimate a pulse and see if it's plausible. This would augment the system's context-awareness (another modality – physiological – to improve confidence in detection). Because this requires very high-quality video to be effective, it might be offered as a feature for high-resolution content and might become more relevant as deepfakes become more life-like.
- **Extend to Images & Other Media:** Currently focused on video, the framework can be extended to image deepfakes (like face swaps in single images) by using similar CNN techniques. Also, consider integrating text deepfake detection (for generated text or deepfake audio in isolation) to evolve into a **multimedia integrity platform**. The architecture's modular nature should allow plugging in an image detector or an audio-only verifier under the same umbrella. This would increase the product's utility (one platform for many deepfake forms) and market reach.
- **Improving Explainability:** Stay updated with XAI techniques. For instance, implement **SHAP values or LIME** for the models to provide an alternative explanation approach (the mentioned UK Home Office challenge winner used SHAP for image deepfakes). This could help identify which features (e.g. eyes, mouth shapes) are most influential on a classification across many videos, thus highlighting common deepfake artifacts. Also, consider user feedback on explanations – maybe develop a simplified “trust score” or integrate a second opinion (ensemble of models) to increase user confidence.
- **Scaling and Optimization:** As usage grows, optimize infrastructure costs. Possibly move heavy inference to serverless GPU services or use **ONNX/TensorRT** to speed up models. Implement caching of results for videos that have been analyzed before (with user consent), to avoid repeated computation. Also, to support enterprise clients, consider offering an **on-premise version** (some companies might want to run the detection in-house for privacy, which means packaging the system in a form that can be deployed to their environment, e.g. a Docker Compose or Helm chart).
- **Responsible AI and Compliance:** Continue conducting **bias and fairness audits** on the detection model. For example, ensure the model is not systematically worse on certain ethnic groups or voice accents; if biases are found, address them via additional training data or model adjustments. Keep up with legal requirements – for instance, deepfake-related laws might require

specific user consent or disclosure; ensure the service complies (e.g. by watermarking all sandbox outputs by default, which may even become legally mandated). Also, as deepfake generation gets more regulated, our sandbox might need eligibility checks (maybe only allow certain content or integrate content moderation).

Milestones (Phase 9): Regular updates (e.g. “Version 2.0” with added physiological module, “Version 3.0” expanding to image deepfakes, etc.). Success in this phase means the product remains effective against new deepfake techniques and continues to gain user trust. In an academic sense, this forward-looking plan shows awareness of the dynamic nature of the deepfake arms race and a commitment to **responsible AI deployment** over time.

Tech Stack Summary

To summarize, the implementation will primarily use **Python** for model development (leveraging PyTorch, TensorFlow, OpenCV, etc.), and **React/Next.js** for the frontend. The system will be deployed via **cloud services** (Vercel for frontend, Google Cloud/AWS for backend) with scalability in mind. The table below outlines the recommended stack and tools for each component:

Component	Technology	Notes
Frontend Web App	Next.js (React), Tailwind CSS	Modern UI framework for interactive pages; Tailwind for styling for fast development.
Backend API	FastAPI (Python)	Handles upload, inference requests, user auth; chosen for high performance async and easy integration with ML libs.
ML/DL Frameworks	PyTorch, PyTorch Lightning	PyTorch for model implementation (Xception, ViT, etc.), Lightning for organized training loops if needed. Some TensorFlow/Keras may be used for certain pre-trained models (with careful integration).
Data Processing	OpenCV, librosa, FFmpeg	OpenCV for frame extraction and image preprocessing; librosa/FFmpeg for audio feature extraction (MFCCs, etc.).

Component	Technology	Notes
Explainability	Captum (for PyTorch)	Library to compute Grad-CAM and other attribution on PyTorch models, simplifying integration of heatmaps.
Metadata analysis	ExifTool, c2pa Python SDK (if available)	ExifTool for general metadata; the C2PA SDK or API for reading content credentials.
Deepfake Generation	Pre-trained models (e.g. FaceSwap, SimSwap, Wav2Lip)	Use as needed in sandbox to avoid training generative models from scratch (licensed carefully). Possibly run in a separate container or process for safety.
Database	PostgreSQL (SQL) or Firebase (NoSQL)	Stores user data, subscriptions, and can log analysis results (for user history or analytics). PostgreSQL is reliable for relational data; Firebase if real-time updates or simpler integration with frontend if needed.
Cloud Deployment	Docker containers, Google Cloud Run / AWS ECS, Cloud Storage (S3/GCS)	Containerization ensures consistency. Use managed services for auto scaling. Storage for user files and model files (e.g. store large model weights on a volume or bucket).
CI/CD	GitHub Actions / GitLab CI	Automate testing (linting, running sample analysis) and deployment to cloud on new commits.
Monitoring & Logging	Prometheus & Grafana (or Cloud-native monitoring)	Track performance metrics, API response times, GPU usage. Set up alerts for failures.
Payment Integration	Stripe API	Manage subscriptions, billing, and restrict features accordingly. Ensure secure handling of payment data (mostly via Stripe's hosted components).

This stack ensures that the system uses **current best practices** in web development and ML deployment (as of 2025). For instance, FastAPI and PyTorch remain industry standards for robust AI APIs, and Next.js on Vercel allows quick global deployments and easy SSR for SEO. The use of Docker and cloud services addresses scalability from day one, critical if this project were to transition to a startup product.

Three-Tier Subscription Plan

Below is an outline of the planned **Free / Pro / Business** tiers, aligning features to user segments and value, following common SaaS tiered pricing strategies:

Plan	Free Tier (Community)	Pro Tier (Professional)	Business Tier (Enterprise)
Target Users	Casual users, students, public interested in verifying a few videos.	Power users, journalists, content creators needing regular deepfake checks and educational use.	Organizations (media companies, social platforms, forensic teams) with large-scale or automated needs.
Detection Features	- Video/image deepfake analysis (web UI) • Basic report: real/fake verdict with confidence • Grad-CAM heatmap on one sample frame • Standard metadata check (camera info) - Full report: detailed explainability (heatmaps on multiple frames, attention insights) • All forensic indicators (blink rate, lip-sync alerts, emotion mismatch analysis) • Priority processing (faster turnaround than Free) - API Access: ability to send content via REST API for integration into workflows • Bulk analysis: handle high volumes with rate limit lifted or separate queue • Custom model options: possibility to fine-tune on client-provided data for higher accuracy (if needed) • Dedicated account manager/support Sandbox Features <i>Not available</i> (Free users cannot generate deepfakes, to prevent misuse at scale) Deepfake Sandbox access: can create limited-length deepfake videos with watermarking (for research/learning). E.g., 10 sandbox generations per month. Advanced Sandbox/Customization: more powerful generation tools (if appropriate) and possibly the option for on-premise sandbox for internal training. Could remove watermarks for sanctioned internal use (with legal agreements). Limits - Upload limit: e.g. 50MB or 30 seconds per video • Usage: e.g. 5 analyses per day		

- Community support (forum or documentation, no direct support)
| - Higher upload limit: e.g. 200MB or 2 minutes per video
- Usage: e.g. 100 analyses per month (with option to purchase more)
- Email support for issues
| - Flexible limits: e.g. 500+ analyses per month or unlimited within fair use
- Analysis of longer videos (5+ minutes) in segments
- SLA with uptime guarantee for API
- Priority support (24/7) |
| **Pricing** | \$0 (Freemium) – Requires account registration for tracking usage. |
\$XX/month (e.g. \$29/mo) – Priced for individuals/small teams. Possibly offer a
discount for annual billing. | Custom or \$YYY/month – Enterprise pricing, likely
tiered by volume. Option for a trial and then custom contract. |

(The above feature limits are examples; actual numbers would be refined based on server capacity and user feedback during beta testing.)

This tiered model follows best practices by **aligning with customer needs**: a free tier to drive adoption and improve the model (via more data and feedback), a pro tier to monetize enthusiasts and professionals, and a high-end tier to engage enterprise clients who derive significant value. It's important that each tier offers a compelling upgrade path: for instance, a free user sees value in Pro's added analysis details and sandbox, while a Pro user in a growing organization sees value in upgrading to Business for API automation and volume scaling.

Responsible AI Note: All tiers, especially Free, will include user reminders about ethical use (e.g. a checkbox before analysis "I confirm I have rights to this video and understand results are probabilistic"). The service will not store or share user-uploaded content beyond analysis, preserving privacy. Sandbox outputs are watermarked by default in all tiers to discourage misuse in public forums, aligning with emerging watermarking standards for AI-generated media.

Conclusion

This roadmap provides a comprehensive step-by-step guide to building a **context-aware deepfake detection and verification framework** that is both academically rigorous and poised for real-world impact. By breaking the development into clear phases – from initial research and model development to full-stack deployment and commercial rollout – we ensure that the project remains manageable and focused at each stage. Key technical decisions, like using a hybrid CNN-Transformer approach and integrating multimodal cues, reflect the latest best practices in deep learning for deepfake detection. Equally, the inclusion of explainability, provenance checks, and a

user sandbox demonstrates a commitment to **user trust and education**, hallmarks of responsible AI deployment.

In 2025, deepfakes are more realistic and widespread than ever, but so are the tools to combat them. Our system leverages cutting-edge research (e.g. ViT, MFCLIP, sentiment analysis) and aligns with industry standards (like C2PA for media authenticity) to stay ahead of deepfake creators. At the same time, we design for scalability and maintainability, using modular architecture and cloud services so the framework can grow from a university capstone project into a full-fledged startup product. By outlining a 3-tier SaaS model, we also address the **viability and sustainability** of the project beyond the academic environment, recognizing the need to support continued development and infrastructure through a business model.

Finally, we emphasize that the fight against deepfakes is ongoing – the roadmap’s later phases prepare for future enhancements like physiological signal analysis and continuous model updates, because maintaining an effective defense will require adaptability as AI synthesis techniques evolve. With this roadmap, the development team can systematically build a deepfake detection platform that is **robust, transparent, and user-centric**, helping restore trust in digital media by quickly distinguishing the “Reel” from the “Real.”